TRABAJO PRÁCTICO Nº 3

OBJETIVOS

- Memoria dinámica y reserva de memoria
- Estructuras de datos dinámicos

1) Copie el siguiente enlace en su navegador: https://classroom.github.com/a/0S0Ctid5 para crear el repositorio donde subirá el **Trabajo Práctico Nro. 3**, realice los pasos ya aprendidos para *clonar* el repositorio en su máquina y poder comenzar a trabajar de forma *local*.

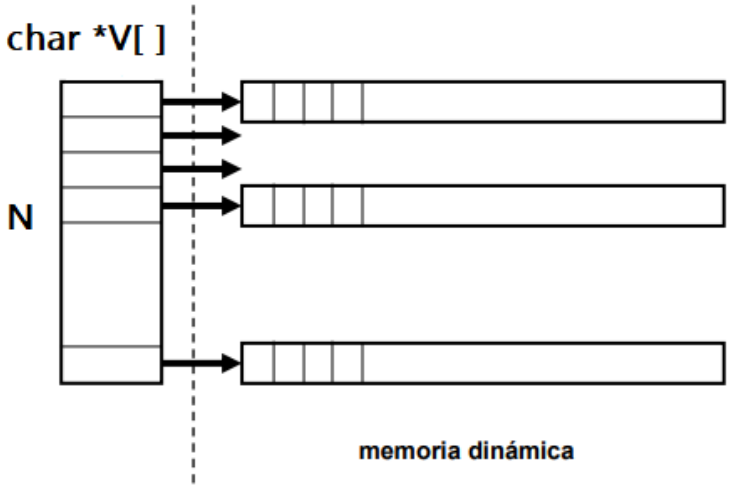
   Nota. No se olvide de incluir el archivo .gitignore en la raíz del repositorio para excluir los archivos .exe, .obj y .tds del mismo.

2) Desarrolla un programa en C que cumpla los siguientes requisitos (nombre del archivo ArregloDeNombres.c):

   Antes de comenzar a desarrollar el programa lea con atención las notas 1, 2 y 3 que acompañan este enunciado y sirven como referencia para resolver el problema.

Parte 1: Ingreso y Listado de Nombres

1. El programa deberá pedir al usuario que ingrese 5 nombres de personas. Estos nombres se almacenarán en un vector donde cada elemento sea un puntero a un bloque de memoria (de tipo `char`) asignado dinámicamente para contener cada nombre.



2. Implementar una función llamada `MostrarPersonas` que reciba el vector y muestre todos los nombres almacenados en él por pantalla.

Parte 2: Implementar la Funcionalidad de Búsqueda utilizando Ramas en git (branch)

1. **Branch para Búsqueda por ID:**
   - Creá una nueva branch llamado `busca-nombre`.
   - En este branch, implementar una función `BuscarNombre` que reciba un número entero positivo de ID (índice del vector) como entrada y Escribir por pantalla el nombre ubicado en esa posición dentro del vector. Si el número ingresado no pertenece al vector escribir “no se encontró el valor buscado”.

2. **Búsqueda por por Id (Palabra Clave) en Main:**
   - Volver al branch principal (`main`).
   - Implementar *una nueva versión* de la función `BuscarNombre`. Esta versión deberá recibir una palabra clave como argumento y retornar el primer nombre dentro del vector que contenga dicha palabra. Si no existe, debería devolver -1. *(Investigá el uso de la función `strstr` para esta implementación.)*

3. **Integración y Resolución de Conflictos:**
   - Realizar un merge entre el branch `busca-nombre` y el branch `main`.
   - A causa de esto, se generará un conflicto entre las ramas y se deberá resolver durante el proceso de merge. (ver Guia uso Merge en Git utilizando VS Code)
   - Resolver el conflicto asignando nombres distintos a cada función, por ejemplo, `BuscaNombrePorId` y `BuscaNombrePorPalabra`

Parte 3: Interfaz de Usuario

1. **Interfaz de Consulta:** Agregá una interfaz de usuario interactiva al programa principal. Esta interfaz debe permitir al usuario ingresar un ID o una palabra clave.
   a. El menú debe contener dos opciones: opción 1 para buscar por ID y opción 2 para buscar por nombre.
2. **Mostrar Resultados:** Según la elección del usuario (búsqueda por ID o palabra clave), el programa deberá utilizar la función correspondiente y mostrar el resultado (el nombre encontrado) en la pantalla."

Nota 1: La librería más común para trabajar con cadenas de caracteres es `string.h`. Algunas de sus funciones más importantes son:

• `strlen(<cadena>)` **:** Devuelve la longitud de la cadena sin tomar en cuenta el caracter de final de cadena.
• `strcpy(<cadena_destino>, <cadena_origen>)` **:** Copia el contenido de *<cadena_origen>* en *<cadena_destino>*.
• `strcat(<cadena_destino>, <cadena_origen>)` : Concatena el contenido de *<cadena_origen>* al final de <>.
• `strcmp(<cadena1>, <cadena2>)` : Compara las dos cadenas y devuelve un 0 si las dos cadenas son iguales, un número negativo si *<cadena1>* es menor que *<cadena2>* y un número positivo (mayor que cero) si *<cadena1>* es mayor que *<cadena2>*

Nota 2: Cuando trabaje con cadena de caracteres (punteros a char) tenga en cuenta que primero tiene que hacer la reserva de memoria según la cantidad de caracteres que quiera guardar y luego utilizando la función *strcpy()* para copiar el contenido. Cuando se cargue por pantalla puede utilizar un arreglo auxiliar (Buff) de tipo char de 50 elementos, para leer los datos temporalmente y luego guardar en la variable asociada.

Nota 3:

Para cargar un arreglo de caracteres con una “frase” use el comando gets(<cadena>)
Tenga en cuenta que para mostrar en pantalla un arreglo de caracteres podemos hacerlo de diversas maneras:

- Dentro de un bucle, desde el primer caracter (indice 0) hasta el último carácter (lo que nos devuelve la función strlen):

```
for(i=0; i<strlen(cadena); i++)
    printf("%c",cadena[i]);
```

- Utilizando el carácter de conversión %s:

```
printf("%s",cadena);
```

- Utilizando puts:

```
puts(cadena);
```

Ejemplos resueltos

```
//Ejemplos resueltos
//Ejemplo 1 - Trabajando  con cadena de caracteres
//Cargar cadena con gets()
//Imprimir frases por pantalla con printf() y puts()

#include <stdio.h>
#include <stdlib.h>
int main() {
    char frase [100];
    char texto[] = "Ud escribió:";
    printf ("Escriba una frase: ") ;
    gets (frase);
    printf ("%s\n", texto) ;
    printf ("%s\n", frase) ;//forma 1 de escribir
    puts (frase);//forma 2 de escribir
    return 0;
}

//Ejemplo 2 - Trabajando  con cadena de caracteres con reserva de
memoria

#include <stdio.h>
#include <stdlib.h>

int main() {
 char *Buff;
   Buff= (char *) malloc(100*sizeof(char));
   printf ("Ingrese una frase\n");
   gets(Buff);
   printf ("%s\n", Buff) ;//forma 1 de escribir
   puts (Buff);//forma 2 de escribir
   free(Buff);
   return 0;
}

//Ejemplo 3 - reserva dinámica de memoria con variable auxiliar

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
 char *Buff; //variable auxiliar
 char *Nombre;
 char *Apellido;
```

```
   Buff= (char *) malloc(100*sizeof(char));
   printf("Ingrese sus nombres: ");
   gets(Buff);

   Nombre= (char *) malloc((strlen(Buff)+1)*sizeof(char));
   strcpy(Nombre,Buff);

   printf("Ingrese sus Apellidos: ");
   gets(Buff);
   Apellido= (char *) malloc((strlen(Buff)+1)*sizeof(char));
   strcpy(Apellido,Buff);

   printf("escribiendo usando printf\n");
   printf ("%s\n", Buff) ;
   printf ("%s\n", Nombre) ;
   printf ("%s\n", Apellido) ;

   printf("escribiendo usando puts\n");
   puts (Buff);
   puts (Nombre);
   puts (Apellido);
//libero la memoria reservada en todos los punteros
  free(Buff);
  free(Nombre);
  free(Apellido);

  return 0;
}
```

3) Considere la siguiente situación: En una distribuidora necesita implementar la carga de productos de preventistas, los cuales recolectan los productos que sus clientes van necesitando. El sistema que se utiliza en la empresa es desarrollado por equipos de programadores donde cada equipo se encarga de una tarea específica. Usted forma parte del equipo de programación que se encargará de hacer el módulo para los preventistas:

Tareas:

Cada preventista puede visitar hasta 5 clientes, los cuales por cuestiones operativas solo puede pedir hasta 10 productos.

Las estructuras de datos necesarias son las siguientes:

```
char *TiposProductos[]={"Galletas","Snack","Cigarrillos","Caramelos","Bebidas"};
struct Producto {
            int ProductoID;       //Numerado en ciclo iterativo
            int Cantidad;         // entre 1 y 10
            char *TipoProducto;   // Algún valor del arreglo TiposProductos
            float PrecioUnitario; // entre 10 - 100
};
struct Cliente {
       int ClienteID;               // Numerado en el ciclo iterativo
       char *NombreCliente;         // Ingresado por usuario
       int CantidadProductosAPedir; // (aleatorio entre 1 y 5)
       Producto *Productos     //El tamaño de este arreglo depende de la variable
                                    // "CantidadProductosAPedir"
};
```

i) Desarrollar una interfaz por consola donde se solicite al usuario la cantidad de clientes.

ii) Solicitar al usuario la carga de los clientes creados dinámicamente en el paso anterior.

iii) A medida que se dé de alta cada cliente, Generar aleatoriamente la cantidad de productos asociados al cliente y sus características.

Ej: producto cargado nro. 2

```
Producto {
    ProductoID=2
    Cantidad = 1;
    *TipoProducto = "Snack";
    PrecioUnitario = 100;
  }
```

NOTA: Investigar cómo generar un valor aleatorio en C.

iv) Implemente una función que calcule el costo total de un producto. Esta función debe recibir como parámetro el producto y devolver el resultado de calcular la Cantidad por el PrecioUnitario.

v) Mostrar por pantalla todo lo cargado. Incluyendo un total a pagar por cliente (sumatoria del costo de todos los productos)

4) Modifique el ejercicio nro 3 para que, en lugar de ingresar un número fijo de nombres, el usuario pueda indicar previamente la cantidad de nombres que ingresará a continuación.

Uso de wiki en el proyecto:

5) ¿Qué es una wiki?¿Para que suele usarse? Investigue, cómo usar la wiki del github. Cargue en la wiki de su repositorio todas las respuestas.

6) En la Wiki de su repositorio: explique con sus propias palabras cuál es la diferencia entre memoria dinámica y estática. ¿Cuáles son las características de cada una? ¿En qué casos se debe usar cada una?